

PROXMOX AUTOMATION SERIES · PHASE
1 FIELD GUIDE

Zero-Touch VM Provisioning on Proxmox VE

Build a Terraform-managed golden template, clone VMs with cloud-init networking on shared NFS storage, and hand them over to Ansible: a repeatable pipeline that turns a manual, console-driven routine into one command.

PVE 9.2 · TERRAFORM 1.15 · BPG/PROXMOX 0.111.1

UBUNTU 26.04 LTS · ANSIBLE-CORE 2.20

AUGUST 2026 · COMPANION REPO: PROXMOX-AUTOMATION

GOLDEN TEMPLATE · CLOUD-INIT · TERRAFORM ·
ANSIBLE

Contents

1. What Phase 1 Delivers	1
2. Prerequisites and Pre-Flight Checks	3
3. Step 1: Prepare the NFS Storage	5
4. Step 2: API Token for Terraform	7
5. Step 3: Build the Control Node	8
6. Step 4: Repository Walkthrough	13
7. Step 5: First Apply	17
8. Step 6: Verification	24
9. Step 7: Ansible First Contact	27
10. Troubleshooting Matrix	30
11. Production Notes and Phase 2 Preview	32

1. What Phase 1 Delivers

This guide is the first module of the Proxmox Automation Series. It turns a manual, console-driven VM deployment routine into a repeatable pipeline: one Terraform command builds a golden template, clones a workload VM onto shared NFS storage, applies cloud-init networking, and hands the machine to Ansible.

The starting point is the daily reality of many infrastructure teams. A fresh server begins as a manual clone of an Ubuntu template, followed by IP, netmask and gateway typed in at the console, SSH logins for every agent installation, hand-edited configuration files, and a final reboot. At twenty deployments a day this is tedious; at ninety it is unmanageable. Phase 1 removes the first and most painful layer of that manual work: the operating system deployment itself. Later phases build on this foundation to automate data disks (Phase 2), security and monitoring agents (Phase 3), and a self-service form (Phase 4).

Everything in this phase runs against the existing three-node homelab cluster (pve-a, pve-b, pve-c with a QDevice) on top of the NFS shared storage introduced in Module 2 of the cluster series. Because the PVE nodes have small local disks, all images, templates and VM disks live on the NFS share; the local-lvm storage of each node stays untouched. The same layout maps directly to a production environment where a SAN or NAS presents shared storage to the cluster.

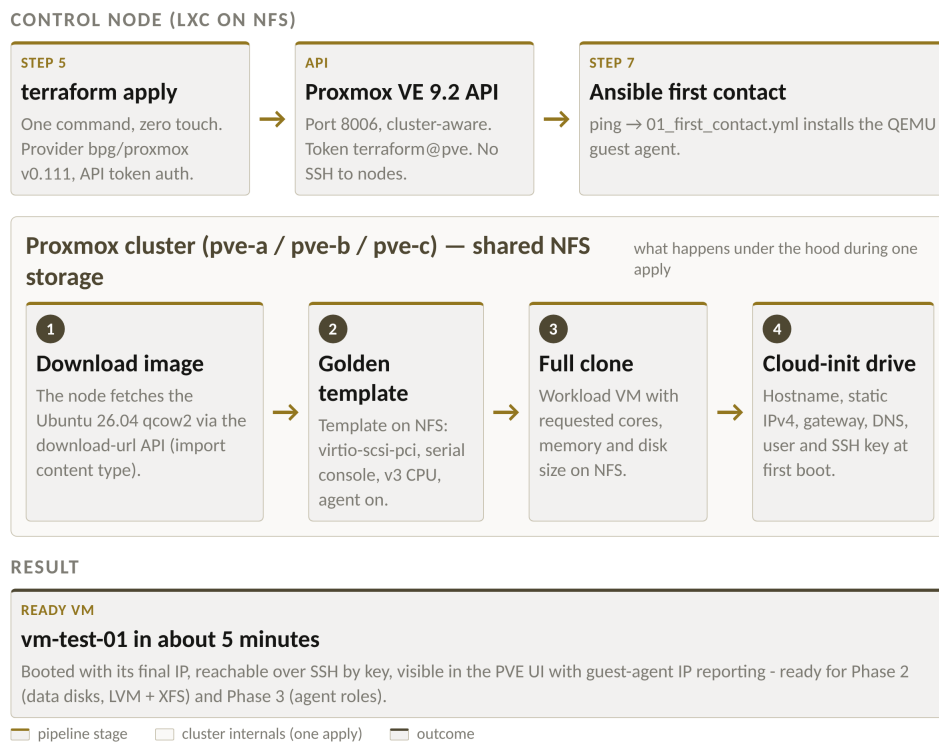


Figure 1. The Phase 1 pipeline: what one terraform apply does.

Verified software baseline

Infrastructure tooling moves quickly, and stale tutorials are the main source of wasted lab hours. Every version referenced in this guide was verified against upstream sources at writing time (August 2026). If your cluster still runs Proxmox VE 8.4, upgrade before starting: that branch stops receiving security updates at the end of August 2026.

Component	Verified version	Why it matters
Proxmox VE	9.2 (May 2026)	Current stable branch; 8.4 reaches end of life on 2026-08-31
Terraform	1.15.x	Stable line; 1.16 is still at release-candidate stage
bpg/proxmox provider	~> 0.111.1	Uses the new proxmox_download_file resource; the old download_file name is deprecated
Ubuntu image	26.04 LTS (Resolute)	Two variants: baseline amd64 and amd64v3 (needs x86-64-v3 CPU)
Ansible	core 2.20.1 (Ubuntu repo)	Only builtin modules are needed in Phase 1; the distro package is fine

Table 1. Version matrix verified against upstream registries and vendor pages.

Scope and boundaries

Phase 1 is deliberately narrow so that each moving part can be verified in isolation. The table below draws the line between what this phase delivers and what the later phases add. Resist the temptation to skip ahead: the disk-mapping work of Phase 2 and the agent roles of Phase 3 both assume the golden template and first-contact playbook built here.

In scope for Phase 1	Deferred to later phases
API token and least-privilege access for Terraform	IPAM integration (NetBox) for automatic address allocation
NFS storage preparation with all required content types	VLAN trunks on physical switches (homelab uses a NAT bridge)
Golden Ubuntu 26.04 template, fully managed by Terraform	Multi-disk layouts, LVM volume groups, XFS filesystems (Phase 2)
One workload VM cloned with cloud-init networking	Zabbix, Trend Micro, ManageEngine, QRadar rsyslog agents (Phase 3)
Ansible first contact: connectivity plus guest agent	Semaphore UI form-driven pipeline and phone_home hook (Phase 4)

Table 2. What this guide builds, and what it intentionally leaves for later.

2. Prerequisites and Pre-Flight Checks

Ten minutes of checks before the first apply saves an evening of debugging afterwards. This chapter walks through the four pre-flight gates: cluster version, CPU instruction set, NFS storage, and network planning. Each check has a single command and a clear pass or fail result, in the spirit of the cluster series where every claim is verified with real output.

2.1 Cluster and host requirements

Requirement	Expected state
Proxmox VE cluster	9.2 verified in the lab; 8.4 is the hard minimum. The import content type used for cloud-image downloads is a PVE 8.4+ feature - on 8.0-8.3 the first apply fails no matter how correct the Terraform code is (Chapter 7.2)
Shared NFS storage	Mounted on every node, visible in Datacenter > Storage
Free NFS capacity	At least 25 GB: image (1 GB) + template (10 GB) + first VM (10 GB)
Network plan	One free static IP for the VM, one for the control container, gateway and DNS noted
Workstation access	SSH or console access to one PVE node and to the NFS-backed network

Table 3. Prerequisites in one view.

The version row is not cosmetic. The `bpg` provider `v0.111` gates the **import** storage content type behind Proxmox VE 8.4.0 (it arrived in 8.1 and was only stabilized in 8.4), and PVE releases before 8.4 additionally reject `.qcow2` file names in the `download-url` API. A node running 8.0.x or 8.2.x passes `terraform plan` perfectly - the failure only appears ten minutes into the first apply, which is exactly how the lab learned this lesson and why Chapter 7.2 documents the upgrade path in full. Run `pveversion now`; if it prints anything below 8.4, budget twenty minutes for the in-place upgrade before continuing.

2.2 The CPU instruction set question (Ubuntu 26.04)

Ubuntu 26.04 LTS ships two cloud image variants. The baseline **amd64** image runs on any x86-64 CPU. The **amd64v3** variant is compiled for the x86-64-v3 microarchitecture level (AVX2, BMI2, FMA, MOVBE) and performs noticeably better on modern hardware; public clouds already default to it. The choice matters because a v3 image on a CPU that lacks those features fails at boot with a kernel panic, and the same failure appears when the virtual CPU type exposed by Proxmox is too old even if the physical host is new. The check below runs on the machine that physically executes the VMs; in the nested-virt lab that is the laptop, not the PVE guest.

RUN ON THE PHYSICAL HOST (THE LAPTOP)

```
# scripts/check-cpu-v3.sh
OK: CPU exposes all x86-64-v3 features (avx2 bmi2 fma movbe).
You can use the amd64v3 cloud image.
```

If the script reports missing features, switch the `cloud_image_url` variable to the baseline image later in Step 4; nothing else in the pipeline changes. Independent of the physical CPU, the VM CPU type must advertise v3 features to the guest. The Terraform code therefore pins **x86-64-v3** instead of the Proxmox default `x86-64-v2-Auto`, a detail covered again in the troubleshooting matrix at the end of this guide.

2.3 Storage and network pre-flight

RUN ON ANY PVE NODE

```
pveversion # expect: pve-manager/9.2/...
hostname -I # note this IP: it becomes pve_endpoint
pvesm status # the NFS storage must be listed as active
pvesm list nfs-shared # current usage on the share
grep net0 /etc/pve/qemu-server/100.conf # bridge of the existing VMs
ping -c2 192.168.122.1 # gateway reachable from the node
```

PRE-FLIGHT OUTPUT FROM THE FIELD RUN

```
pveversion
pve-manager/9.2.2/b9984c6d90a4bd80 (running kernel: 7.0.2-6-pve)

hostname -I
192.168.122.11

ping -c2 192.168.122.1
PING 192.168.122.1 (192.168.122.1) 56(84) bytes of data.
64 bytes from 192.168.122.1: icmp_seq=1 ttl=64 time=0.312 ms
64 bytes from 192.168.122.1: icmp_seq=2 ttl=64 time=0.298 ms

--- 192.168.122.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
```

The pveversion line above is the actual field run: the cluster had been upgraded from 8.0.0 to 9.2.2 before provisioning succeeded, which is why Chapter 7.2 devotes a full section to that upgrade. The hostname -I output is the value that becomes pve_endpoint in terraform.tfvars in Step 5, prefixed with https and suffixed with port 8006. And the two ping replies with zero packet loss close the network gate: the node sees the same gateway the future VMs will use as their default route.

Note the storage ID shown by pvesm status; it is one of the values you will copy into terraform.tfvars in Step 5. Pick the workload IP now as well, and make sure it is outside any DHCP range the lab router serves. Static cloud-init addresses and DHCP pools that quietly overlap are a classic source of intermittent connectivity problems that look like provisioning bugs but are not.

3. Step 1: Prepare the NFS Storage

The PVE nodes carry small local disks, so the NFS share carries everything: the downloaded cloud image, the golden template, the workload VM disks, and later the snippet files that carry custom cloud-init payloads. Proxmox storages declare which content types they accept, and a freshly created NFS storage usually only enables disk images. Two content types in particular are easy to miss and produce confusing errors later: **Import**, without which the Terraform download resource fails with an HTTP 500, and **Snippets**, which Phase 4 will need for custom user-data files.

3.1 Enable the required content types

In the web interface, open Datacenter, then Storage, select the NFS storage, click Edit, and tick every entry in the Content list. The CLI equivalent below achieves the same in one line and is the version to keep in the lab journal alongside its output, matching the style of the cluster series.

RUN ON ANY PVE NODE

```
pvesm set nfs-shared -content images,rootdir,vztmpl,snippets,import
```

```
pvesm status
```

```
Name Type Status Total(KiB) Used(KiB) Available(KiB) %
local dir active 17687844 4986852 11777140 28.19%
local-lvm lvmthin active 10940416 0 10940416 0.00%
nfs-shared nfs active 486256640 197097472 286114816 40.53%
```

Content type	Purpose in this pipeline
images	VM disks: golden template and workload clones
rootdir	Container volumes: the Ansible control node LXC
vztmpl	Container templates downloaded via pveam
snippets	Custom cloud-init user-data files (used from Phase 4 on)
import	Storage target for the download-url API used by proxmox_download_file

Table 4. Content types required on the NFS storage.

The Import content type deserves a short explanation because it is new relative to most tutorials. Proxmox VE exposes a download-url endpoint that lets the node itself fetch a file over HTTPS directly into a storage. The bpg provider wraps that endpoint in the proxmox_download_file resource, which is why Step 5 needs no wget on the control node and no qm importdisk at all. The endpoint refuses to write into a storage that does not declare the import content type, and that refusal surfaces in Terraform as a generic 500 error; fixing the storage configuration resolves it.

4. Step 2: API Token for Terraform

The Terraform provider needs credentials for the Proxmox API. Three methods exist: username with password, an auth ticket, or an API token. Tokens are the right choice for automation because they are revocable one by one, they carry their own permission boundary, and they never embed a human password in a CI variable. The bootstrap script in the companion repository creates a dedicated user and token in three idempotent steps; run it once on any node, because user and permission objects live in the cluster-wide configuration database and replicate to every node automatically.

SCRIPTS/PVE-BOOTSTRAP.SH (RUN ON ANY PVE NODE)

```
pveum user add terraform@pve --comment "Terraform automation"
pveum acl modify / --users terraform@pve --roles Administrator
pveum user token add terraform@pve provider --privsep 0
```

output (uuid shown only once):

```
#
# | token-name | value | expire | privsep |
# |-----|-----|-----|-----|
# | provider | 1a2b... | never | 1 |
#
```

The token value is the concatenation of the full token ID and the UUID separated by an equals sign, for example `terraform@pve!provider=1a2b3c4d-....`. Copy it exactly; leading or trailing whitespace is the most common reason a freshly created token is rejected. For interactive use the value goes into `terraform.tfvars`, which the repository `.gitignore` deliberately excludes from version control. For CI the same value belongs in the `PROXMOX_VE_API_TOKEN` environment variable, and the `tfvars` file stays empty.

For the lab, Administrator on the root path is acceptable and keeps the guide focused. A production hardening pass should replace it with a custom role: the download resource documents `Sys.Audit`, `Sys.Modify` and `Datastore.AllocateTemplate` as its minimum requirements, to which VM creation adds `VM.Allocate`, `VM.Clone` and related privileges. The token in this guide uses privilege separation disabled (`privsep 0`) so the token inherits the user rights; enabling `privsep` is part of the same hardening pass.

5. Step 3: Build the Control Node

The control node is a small LXC container on the NFS storage that runs Terraform and Ansible and owns the repository checkout. A container is the right shape for this job: it boots in seconds, consumes roughly 100 MB of memory when idle, and its root filesystem lands on rootdir content already enabled in Step 1. An unprivileged container is fine because nothing in Phase 1 requires kernel-level privileges; nesting stays disabled until Phase 4 installs Semaphore, which itself runs as a container inside the control node only if you choose that deployment style.

5.1 Create the container

RUN ON ANY PVE NODE

```
# pick the current Ubuntu template from the catalog
pveam available --section system | grep ubuntu
pveam download nfs-shared ubuntu-26.04-standard_26.04-1_amd64.tar.zst

pct create 900 nfs-shared:vztmpl/ubuntu-26.04-standard_26.04-1_amd64.tar.zst \
--hostname ctrl-01 --memory 2048 --cores 2 --rootfs nfs-shared:8 \
--net0 name=eth0,bridge=vbr0,ip=192.168.122.10/24,gw=192.168.122.1 \
--unprivileged 1 --features nesting=1 --start 1
```

The exact template version string evolves; what matters is that the template is downloaded to the NFS storage (so every node can start the container after a migration) and that the container gets a static address on the same network as the future VMs, because Ansible will reach them from here. Attach it to the same bridge the pre-flight grep in Chapter 2 revealed, vbr0 in the lab examples. Give the container 2 GB of memory; Terraform state operations and image-intense plans are comfortable at that size, and the host overcommit on a three-node laptop lab absorbs it easily.

THE SAME STEPS, WITH THEIR OUTPUT

```
# pveam download nfs-shared ubuntu-26.04-standard_...amd64.tar.zst
downloading ubuntu-26.04-standard_26.04-1_amd64.tar.zst from
download finished

# pct create 900 ... (prints nothing on success; the container starts)

# pct list
VMID Status Lock Name
900 running ctrl-01
```

Both container commands follow the Proxmox house convention that success is quiet. pveam download prints a progress meter while it fetches the template and ends with download finished; pct create prints nothing at all and simply returns the prompt while the container boots in the background. The

confirmation habit for both is the same: list what exists. `pct list` showing VMID 900 as running is the gate to the next section; if the status column reads stopped, run `pct start 900` and read the error it prints before debugging anything deeper.

First login: there is no default password

A freshly created Ubuntu container rejects every console login attempt, root included, and this surprises everyone once. The reason is that the Ubuntu LXC template ships with the root account locked and no default password at all; the template assumes configuration happens from the host, not from a console. The intended entry point is therefore `pct enter`, which opens a root shell inside the container through the kernel namespace without any credentials. Every step labelled run inside `ctrl-01` in the remainder of this guide uses this door, and a console password only becomes necessary if you plan to log in at `tty1` directly.

RUN ON ANY PVE NODE

```
pct enter 900 # root shell inside ctrl-01, no password

# optional, inside ctrl-01: set a console password
passwd root

# non-interactive alternative, run from the node:
pct exec 900 -- bash -c 'echo "root:changeme" | chpasswd'
```

5.2 Install Terraform and Ansible inside

RUN INSIDE CTRL-01 (ROOT SHELL VIA PCT ENTER)

```
# Terraform from the HashiCorp repository (apt universe lags behind).
# Minimal LXC templates ship no gpg binary; modern apt (>= 2.4) reads
# the ASCII-armored key directly when stored with the .asc extension:
wget -qO /usr/share/keyrings/hashicorp.asc \
https://apt.releases.hashicorp.com/gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp.asc] \
https://apt.releases.hashicorp.com $(lsb_release -cs) main" \
> /etc/apt/sources.list.d/hashicorp.list
apt-get update && apt-get install -y terraform

# Ansible from the distribution repository
apt-get install -y ansible
```

VERSION CHECKS, EXACTLY AS THE FIELD RUN PRINTED THEM

```
terraform -version
Terraform v1.15.9
on linux_amd64

ansible --version
ansible-core 2.20.1
config file = /etc/ansible/ansible.cfg
executable location = /usr/bin/ansible
python version = 3.13.2 (/usr/bin/python3)
jinja version = 3.1.6
libyaml = True
```

The version banners are worth pasting into the lab journal exactly as printed, because they close the tooling gate with evidence instead of assumptions. Terraform v1.15.9 came from the HashiCorp resolute repository installed minutes earlier, and ansible-core 2.20.1 is what the Ubuntu distribution ships at this point, pulled in by the ansible 13.1.0 meta package. If `ansible --version` instead dies with the locale error below, the container hit the third and last template gotcha before the pipeline could continue.

The locale that was never generated

The first `ansible --version` call in a fresh container can end with **ERROR: Ansible could not initialize the preferred locale instead of a version banner**. The Ubuntu template sets `LANG` to `en_US.UTF-8` in its environment defaults but never generates that locale, and the same omission produced the perl locale warnings during the package installation above. Ansible is stricter than perl and refuses to start altogether. Generating the locale once fixes the warnings and the Ansible error in a single stroke, and the fix survives reboots because `update-locale` writes it into `/etc/default/locale`.

RUN INSIDE CTRL-01 (ONLY IF ANSIBLE --VERSION FAILS)

```
apt-get install -y locales
locale-gen en_US.UTF-8
update-locale LANG=en_US.UTF-8

ansible --version # ansible-core 2.20.1
```

One detail trips up nearly everyone in a container: the commonly documented pipe into `gpg --dearmor` assumes `gnupg` is installed, but the minimal Ubuntu template does not include it. The pipe fails with `command not found`, no keyring is ever written, and the next `apt-get update` rejects the HashiCorp repository as unsigned with a `NO_PUBKEY` notice, because the signed-by reference points at a file that does not exist. The `.asc` form above sidesteps `gnupg` entirely. The alternative is to `apt-get install gnupg` first and then run the classic `dearmor` pipe; both paths end at the same verified repository. Note that the Ubuntu 26.04 codename, *resolute*, has its own distribution on the HashiCorp mirror, so `lsb_release -cs` needs no special-casing.

5.3 Prepare the SSH key material

RUN INSIDE CTRL-01

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""  
cat ~/.ssh/id_ed25519.pub # this line is injected into every VM
```

SSH-KEYGEN OUTPUT, FINGERPRINT SHORTENED

```
Generating public/private ed25519 key pair.  
Your identification has been saved in /root/.ssh/id_ed25519  
Your public key has been saved in /root/.ssh/id_ed25519.pub  
The key fingerprint is:  
SHA256:1Zc5k... root@ctrl-01  
  
cat ~/.ssh/id_ed25519.pub  
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... root@ctrl-01
```

The fingerprint line is the identity of the keypair; the field run printed a value starting SHA256:1Zc5k. The public key line is the string Terraform injects into every cloned VM, so the characters after ssh-ed25519 are what later resurfaces in the qm cloudinit output on the hypervisor and inside the guest in /home/ubuntu/.ssh/authorized_keys. Seeing the same key material in all three places is a quick end-to-end sanity check that Chapter 8 makes use of.

One keypair serves two purposes here. The public key is baked into every provisioned VM through the cloud-init user_account block, so the control node can reach the machines it creates. The same control node later uses the key for git operations when the repository moves to a remote. Note that Phase 1 does not need SSH access from the control node to the PVE nodes themselves; every operation goes through the API token, and SSH to the nodes only becomes necessary when custom snippet uploads arrive with Phase 4.

5.4 Copy the repository into the container

The repository archive starts life on the workstation where it was downloaded. Three hops move it into the container, and each hop uses the transport that segment already offers. From the laptop the cluster is reachable over SSH, because the laptop itself acts as the NAT gateway of the lab network at 192.168.122.1. From the node into the container the canonical Proxmox tool is pct push, which writes through the container filesystem directly and needs no SSH server inside the container at all. Inside the container a plain unzip finishes the job.

WORKSTATION -> PVE-A -> CTRL-01

```
# on the laptop: copy the archive to pve-a (any node works)
scp ~/Downloads/proxmox-automation.zip root@:/root/

# on pve-a: push it into the container (no SSH needed inside)
pct push 900 /root/proxmox-automation.zip /root/proxmox-automation.zip

# inside ctrl-01: unpack into the home directory
apt-get install -y unzip
unzip ~/proxmox-automation.zip -d ~/
ls ~/proxmox-automation
```

THE THREE HOPS, EACH WITH ITS SIGN OF SUCCESS

```
# on the laptop:
proxmox-automation.zip 100% 452KB 9.1MB/s 00:00

# on pve-a: pct push prints nothing on success

# inside ctrl-01:
root@ctrl-01:~# ls ~/proxmox-automation
ansible docs README.md README.tr.md scripts terraform
```

Each hop announces success differently, and knowing the expected behavior of each saves a round of doubt. `scp` prints its progress line and exits quietly once the archive lands on the node; `pct push` returns to the prompt without printing anything, the same silence convention as `pct create`; and the `ls` inside the container is the definitive receipt, because it lists the unpacked repository exactly as the layout diagram in the next chapter shows it. The archive is small, so the whole chain takes seconds even over the lab bridge.

Why not `scp` straight from the laptop into the container? A freshly created Ubuntu container has no root password and no authorized key, so SSH has nothing to authenticate against; the account stays locked until `passwd` or an injected key changes that. `pct push` deliberately bypasses the problem, which is exactly why it is the standard tool for seeding containers with files. Once the repository moves to a git remote, this whole hop chain collapses into a single git clone inside the container, and git is already present because the Terraform package pulled it in as a dependency. That is also the moment the pipeline stops depending on file copying altogether.

6. Step 4: Repository Walkthrough

The companion repository (proxmox-automation) holds the entire Phase 1 pipeline: Terraform for the infrastructure, Ansible for first contact, two helper scripts, and this guide. Clone it onto the control node; the layout below is deliberately flat so that each file maps to exactly one concept. This chapter explains the design decisions encoded in the Terraform files so that when something behaves unexpectedly later, the reason is already documented here.

REPOSITORY LAYOUT

```
proxmox-automation/
├─ terraform/
│  ├─ versions.tf # provider pin ~> 0.111.1
│  ├─ providers.tf # endpoint + API token
│  ├─ variables.tf # every knob, documented
│  ├─ template.tf # image download + template
│  ├─ vm.tf # clone + cloud-init
│  ├─ outputs.tf # VMID, IP, ssh command
│  └─ terraform.tfvars.example
├─ ansible/
│  ├─ ansible.cfg
│  ├─ inventory/hosts.yml
│  └─ playbooks/01_first_contact.yml
├─ scripts/ (pve-bootstrap.sh, check-cpu-v3.sh)
└─ docs/phase-1-guide.pdf
```

6.1 Key design decisions

Decision	Rationale
proxmox_download_file instead of the older resource name	The provider deprecated proxmox_virtual_environment_download_file; it will be removed in v1.0. New code should use the short name.
scsi_hardware = virtio-scsi-pci	Required by Ubuntu cloud images, and the only controller that supports online disk resize, which Phase 2 depends on.
cpu type pinned to x86-64-v3	The PVE default (x86-64-v2-Auto) lacks AVX2, so amd64v3 guests crash at boot. The -Auto variant stays live-migration safe across mixed hosts.
Serial console (serial_device + vga serial0)	Official cloud-image convention; fixes the blank-console symptom in noVNC when a standard VGA is attached instead.
Full clone instead of linked	Independent disks on NFS mirror production DB-server practice; linked clones remain a Phase 5 optimization option.
Native initialization block, no custom snippets	Phase 1 needs only hostname, IP, DNS and user data, all of which the provider models natively; no SSH to PVE nodes required.
Guest agent installed by Ansible, not cloud-init	Keeps user-data snippet-free and puts the agent under configuration management where drift is detected.

Table 5. Decisions worth remembering when something misbehaves.

6.2 The template resource, annotated

TERRAFORM/TEMPLATE.TF (EXCERPT)


```

resource "proxmox_download_file" "ubuntu_image" {
  content_type = "import" # PVE 8.4+; Import on storage
  datastore_id = var.storage_id # the NFS share
  node_name = var.template_node # node that downloads
  url = var.cloud_image_url
  file_name = "ubuntu-2604-clouidmg.qcow2"
  upload_timeout = 3600 # default 600 s is too tight
  overwrite_unmanaged = true # track the /current/ image
}

resource "proxmox_virtual_environment_vm" "ubuntu_template" {
  name = var.template_name
  node_name = var.template_node
  vm_id = var.template_vmid # fixed at 9000
  template = true
  agent { enabled = true }
  cpu { cores = 2
  type = var.cpu_type } # x86-64-v3, see Chapter 2
  scsi_hardware = "virtio-scsi-pci"
  disk {
    datastore_id = var.storage_id
    import_from = proxmox_download_file.ubuntu_image.id
    interface = "scsi0"
    size = var.template_disk_size_gb
  }
  serial_device {}
  vga { type = "serial0" }
  boot_order = ["scsi0"]
}

```

Three details are easy to overlook. First, the `import_from` reference chains the template to the downloaded image, which orders the two resources correctly without an explicit `depends_on`, and it selects the native PVE import-from disk option - pure API-token workflow, no SSH into the node (the older `file_id` attribute, which most tutorials use, silently SSHes in and runs `qm disk import`; Chapter 7.2 shows why that matters). Second, `upload_timeout` raises the provider default of 600 seconds, which is a race against slower uplinks on an 820 MB image. Third, the template itself carries no initialization block; cloud-init settings belong to the clone, so one golden template can serve workloads with completely different network identities. The template is a mould, not a machine.

6.3 The workload clone, annotated

TERRAFORM/VM.TF (EXCERPT)

```
resource "proxmox_virtual_environment_vm" "test_vm" {
  name = var.vm_name
  node_name = var.vm_node
  vm_id = var.workload_vmid # fixed at 200

  clone {
    vm_id = proxmox_virtual_environment_vm.ubuntu_template.vm_id
    full = true
    retries = 3
  }

  agent {
    enabled = true
    wait_for_ip { disabled = true } # see Section 7.3
  }

  disk { # resizes the cloned root disk
    datastore_id = var.storage_id
    interface = "scsi0"
    size = var.vm_disk_size_gb
  }

  initialization {
    datastore_id = var.storage_id # cloud-init drive location
    # no hostname here: provider v0.111 removed it;
    # the VM name (a valid DNS name) becomes the hostname
    dns { domain = var.search_domain
      servers = var.dns_servers }
    ip_config {
      ipv4 { address = "${var.vm_ip_address}/${var.vm_ip_cidr}"
        gateway = var.vm_ip_gateway }
    }
    user_account {
      keys = [trimspace(file(pathexpand(var.ssh_public_key_path)))]
      username = var.vm_username
    }
  }
}
```

The agent block carries one line that earns its place in the excerpt: `wait_for_ip disabled`. Ubuntu cloud images ship without the QEMU guest agent, and the provider, seeing agent enabled, polls the absent agent for network interfaces both after the VM starts and again on every refresh - each poll armed with the fifteen-minute default timeout. Disabling the lookup trades agent-based IP discovery (which a static-IP design does not need) for applies and plans that never stall; Section 7.3 shows the warning that appears when the block is omitted.

The `pathexpand` call in `user_account` exists because Terraform `file()` does not expand a leading tilde on its own; without it the default key path fails with a confusing "no such file" error. The disk block on a clone resizes the inherited root disk, which is why `vm_disk_size_gb` must be greater than or equal to the template disk size. Growing a root disk here is convenience; growing data disks with filesystem expansion behind them is precisely what Phase 2 automates properly.

7. Step 5: First Apply

With the storage prepared, the token in hand and the control node tooled up, the actual provisioning is anticlimactic, which is the point. Copy the example variables file, fill in the values gathered during pre-flight, and let Terraform show the plan before anything is created.

RUN INSIDE CTRL-01

```
cd ~/proxmox-automation/terraform
cp terraform.tfvars.example terraform.tfvars
nano terraform.tfvars # endpoint, token, storage_id, IPs

terraform init
terraform validate # schema check, no cluster contact
terraform plan
```

THE VALUES THAT MATTER IN TERRAFORM.TFVARS (LAB RUN)

```
pve_endpoint = "https://192.168.122.11:8006/"
pve_api_token = "terraform@pve!provider="
storage_id = "nfs-shared"

vm_name = "vm-test-01" # plus vm_ip_address, gateway, dns
workload_vmid = 200 # the rest keeps the example defaults
```

Four values carry the whole run: the endpoint, which is the Chapter 2 hostname -I output prefixed with https and suffixed with port 8006; the token ID and UUID joined by an equals sign exactly as pveum printed them in Step 2; the storage ID the pre-flight revealed; and the SSH public key path from the previous step. Everything else already carries a working default in terraform.tfvars.example, from the template VMID 9000 to the resolute cloud image URL. Keep the filled file out of git; the repository .gitignore already excludes it by name.

TERRAFORM INIT + VALIDATE, FIRST RUN OUTPUT

```
Initializing the backend...
Initializing provider plugins...
- Finding bpg/proxmox versions matching "~> 0.111.1"...
- Installing bpg/proxmox v0.111.1...
- Installed bpg/proxmox v0.111.1 (signed, key ID F0582AD6AE97C188)

Terraform has created a lock file .terraform.lock.hcl to record the
provider selections it made above. Include this file in your version
control repository so that Terraform can guarantee to make the same
selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

terraform validate
Success! The configuration is valid.
```

The key ID printed by init deserves one sentence of trust context: F0582AD6AE97C188 is the signing key the provider maintainer registered with the Terraform Registry, so seeing it confirms the binary came from the registry and not from a tampered mirror. The validate one-liner that follows is the cheapest safety net in the whole toolchain, a full schema check with zero cluster contact. Both lines together are the green light for terraform plan.

The validate step between init and plan is free and worth forming as a habit: it checks every attribute and block in the configuration against the real provider schema without sending a single request to the cluster. The two schema errors most likely to bite anyone following older tutorials, a renamed VLAN attribute and a removed hostname argument, are exactly the class of failure validate catches in under a second, before any credentials are involved.

A healthy plan shows three resources to add: the downloaded image file, the template VM and the workload VM. Nothing should be in the destroy or replace columns on a first run. Review the plan output with the same skepticism applied to cluster commands in the earlier series; if a value looks wrong, it is cheaper to fix the variables now than to rebuild the template after the apply.

TERRAFORM PLAN - THE LINES THAT MATTER

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# proxmox_download_file.ubuntu_image will be created
# proxmox_virtual_environment_vm.ubuntu_template will be created
# proxmox_virtual_environment_vm.test_vm will be created
```

Plan: 3 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ ssh_command = "ssh ubuntu@192.168.122.50"
+ template_vm_id = 9000
+ test_vm_id = 200
+ test_vm_ip = "192.168.122.50"
+ test_vm_name = "vm-test-01"
```

THE FIRST APPLY, CONDENSED FROM THE FIELD TRANSCRIPT

```
terraform apply
```

```
...
```

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

```
proxmox_download_file.ubuntu_image: Creating...
proxmox_download_file.ubuntu_image: Still creating... [4m25s elapsed]
proxmox_download_file.ubuntu_image: Creation complete after 4m25s
proxmox_virtual_environment_vm.ubuntu_template: Creating...
proxmox_virtual_environment_vm.ubuntu_template: Creation complete after 11s
proxmox_virtual_environment_vm.test_vm: Creating...
proxmox_virtual_environment_vm.test_vm: Still creating... [15m00s elapsed]
proxmox_virtual_environment_vm.test_vm: Creation complete after 15m5s
```

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

```
ssh_command = "ssh ubuntu@192.168.122.50"
template_vm_id = 9000
test_vm_id = 200
test_vm_ip = "192.168.122.50"
test_vm_name = "vm-test-01"
```

The timings above are the actual field measurements, and they tell the real story of a first apply. The image download dominates with four and a half minutes, the template assembles in eleven seconds, and

the fifteen minutes attached to the VM are the agent-wait stall that Section 7.3 dissects and the shipped configuration already disables; with `wait_for_ip` disabled the VM line finishes in seconds, which is what every later apply in the series looked like. The Outputs block at the end is Terraform handing over the keys: the VMID pair feeds the hypervisor checks in Chapter 8, and `ssh_command` is ready to run verbatim.

7.1 What happens under the hood

One apply triggers a chain of four operations inside the cluster. Knowing the order makes the apply output readable and explains where the time goes on the first run versus every later run.

- **Image download.** The PVE node itself fetches the qcow2 over HTTPS into the NFS storage using the `download-url` API with the **import** content type (a PVE 8.4+ feature). The repo allows up to `upload_timeout = 3600` seconds, because the provider default of 600 s killed slower lab links mid-download. On a homelab link this step dominates the first apply (roughly 820 MB) and disappears entirely on later runs because the file is already managed.
- **Template assembly.** The provider creates the template VM, imports the downloaded image as `scsi0` through the native PVE `import-from disk` option (no SSH into the node needed), applies the controller, CPU, agent and console settings, resizes the disk to the configured size, and marks the VM as a template.
- **Clone.** The workload VM is created as a full clone on NFS with the requested cores, memory and resized root disk.
- **Cloud-init drive.** The provider generates the cloud-init ISO from the initialization block and attaches it; the VM boots and cloud-init applies hostname, network, user and key on first boot.
- **Start and agent wait.** The provider starts the VM and then polls the QEMU guest agent for network interfaces - a phase that behaves very differently before and after the agent is installed (Section 7.3).

about 5 minutes

Field-measured first apply: 4m25s image download plus 11s template assembly, with the clone finishing in seconds once the Section 7.3 agent-wait fix is in place; applies that only clone a VM run about one minute.

The apply ends with the outputs defined in `outputs.tf`: the template VMID, the workload VMID, the static IP and a ready-to-run `ssh` command. If any resource fails, read the error from the bottom up; Proxmox API errors surface with the failing task UPID, and the matching task log lives in the node Task History in the web interface, which usually names the real cause more precisely than the Terraform wrapper message.

STATE AND OUTPUTS AFTER A GOOD APPLY

```
terraform state list
proxmox_download_file.ubuntu_image
proxmox_virtual_environment_vm.ubuntu_template
proxmox_virtual_environment_vm.test_vm

terraform output ssh_command
"ssh ubuntu@192.168.122.50"
```

Two commands make good closing rituals for the chapter. `terraform state list` names exactly the three resources the plan promised, which is the cheapest consistency check between code and reality; and `terraform output` with a single output name prints just that value, ready to paste into a shell or a journal. The `-raw` variant of the same command even drops the quotation marks, which makes the result directly usable inside scripts and one-line shell loops.

7.2 The version gate: when the node is too old

The first real apply in the lab ended not with the expected outputs but with a ten-minute wait and then this error - reproduced here because the combined message is confusing enough to deserve translation:

TERRAFORM APPLY ON PROXMOX VE 8.0.0

```
proxmox_download_file.ubuntu_image: Still creating... [10m00s elapsed]
|
| Error: Error downloading file from url
| Could not download file 'ubuntu-2604-cloudimg.qcow2', unexpected error:
| error download file to datastore nfs-shared: failed waiting for url
| download: timeout while waiting for task "UPID:pve-a:..." to complete,
| and the content type 'import' is not supported by the Proxmox VE
| version 8.0.0
|
```

Three separate facts are packed into that message. First, the download task itself timed out: the provider waits `upload_timeout` seconds (600 by default) for the PVE task to finish, and a slow uplink fetching an 820 MB image can exceed that alone. Second, the trailing hint is the provider being helpful: it queried the cluster version, saw 8.0.0, and appended that the import content type needs PVE 8.4 or newer. Third - and this is the part no error message mentions - even if the download had succeeded, the original configuration imported the disk via the `file_id` attribute, which silently SSHes into the PVE node and runs `qm disk import`; an API token cannot open an SSH session, so the apply would have failed at the template step anyway. The provider source code makes all three mechanisms explicit, and the fixes are already in this repository.

- **Upgrade the node to PVE 8.4 or newer.** The import content type and `.qcow2` download names both require it; 9.2 is what the lab runs now and what these guides are verified against.
- **`upload_timeout = 3600`.** The repo now sets this on the download resource so slow links do not race a 10-minute fuse.

- **disk import_from instead of file_id.** import_from uses the native PVE import-from API option - pure API-token workflow, no SSH credentials anywhere in Phase 1.

On a fresh lab node with no production VMs, the in-place upgrade is the shortest safe path: bring the node to the latest 8.4 on the bookworm repositories, then hop to 9.2 on trixie. Both hops are ordinary apt full-upgrades; the API user, token, NFS storage and the control container survive both reboots untouched.

RUN ON PVE-A - HOP 1: 8.0.0 TO LATEST 8.4

```
# no-subscription repo for the lab (enterprise repo needs a subscription)
echo "deb http://download.proxmox.com/debian/pve bookworm pve-no-subscription" \
> /etc/apt/sources.list.d/pve-no-subscription.list
sed -i 's/^deb/#deb/' /etc/apt/sources.list.d/pve-enterprise.list 2>/dev/null

apt update && apt full-upgrade -y
pveversion # expect pve-manager/8.4.x
reboot # new kernel; ctrl-01 comes back on boot
```

RUN ON PVE-A - HOP 2: 8.4 TO 9.2

```
# Debian base bookworm -> trixie, PVE repo bookworm -> trixie
sed -i 's/bookworm/trixie/g' /etc/apt/sources.list
sed -i 's/bookworm/trixie/g' /etc/apt/sources.list.d/pve-no-subscription.list

apt update && apt full-upgrade -y
pveversion # expect pve-manager/9.2.x
reboot
```

Two cleanup details close the chapter. The abandoned download task kept running on the node after Terraform gave up waiting, so a partial image file may sit in the storage; the download resource sets `overwrite_unmanaged = true`, which deletes and replaces any such file on the next apply, so no manual `rm` is required. And if a hop is ever skipped and the apply fails again, run `pveversion` before debugging anything else - every symptom in this section had the same root cause, a version number that was easy to check and easy to overlook.

7.3 The fifteen-minute warning that is not an error

The first fully successful apply in the lab had one final surprise left. The image downloaded in four and a half minutes, the template appeared in eleven seconds - and then the workload VM sat at Still creating for exactly fifteen minutes before finishing with this:

TERRAFORM APPLY ON PVE 9.2.2 - THE TAIL OF A GOOD APPLY

```

proxmox_virtual_environment_vm.test_vm: Still creating... [15m00s elapsed]
proxmox_virtual_environment_vm.test_vm: Creation complete after 15m5s

| Warning: error waiting for network interfaces from QEMU agent
|
| with proxmox_virtual_environment_vm.test_vm,
| on vm.tf line 7, in resource "proxmox_virtual_environment_vm" "test_vm":
| 7: resource "proxmox_virtual_environment_vm" "test_vm" {
|
|   timeout while waiting for the QEMU agent on VM "200" to publish the
|   network interfaces
|
|
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

```

The fifteen-minute runtime is the tell: the clone itself finished in seconds, the VM booted almost immediately, and the entire wait was the provider polling the guest agent. The agent block enables the QEMU agent interface in the VM configuration, and the provider - both after starting a VM and on every subsequent refresh during plan and apply - asks the agent for network interface data, retrying until the timeout attribute of the agent block (default 15m) expires. Ubuntu cloud images ship without the agent binary, so every poll fails and the provider eventually gives up with a Warning, not an Error: the VM is running, the state is clean, and the only real loss is that the `ipv4_addresses` attribute stays empty. The verification in Chapter 8 does not depend on it because the IP plan is static.

There are three sane responses, and the repository ships the first. `wait_for_ip { disabled = true }` inside the agent block skips the agent lookup entirely - the right default for this pipeline, which assigns static IPs from `tfvars` and manages the agent through Ansible in Chapter 9 rather than cloud-init. Shortening the agent timeout (for instance to 2m) shrinks the stall instead of removing it, a reasonable middle ground if agent-based IP discovery in the state is wanted later. And doing nothing is also defensible: once Chapter 9 installs and starts `qemu-guest-agent`, the polls succeed in seconds and the warning disappears on its own. What is never correct is interrupting the apply over it - a Ctrl-C mid-create taints the resource and leaves the VMID allocated for a partial cleanup.

7.4 Tearing it down

Every experiment in this series eventually ends with a clean slate, and the same tool that built the stack unbuilds it. `terraform destroy` renders a plan with exactly the three resources in the destroy column, asks for the same yes confirmation, and removes the workload VM, the template and the downloaded image in reverse dependency order. The control container, the API token and the storage configuration are not Terraform resources; they survive the destroy untouched, which is deliberate, because the next apply then needs nothing but itself to rebuild the whole stack.

TERRAFORM DESTROY - EXPECTED SHAPE

```
Plan: 0 to add, 0 to change, 3 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes
proxmox_virtual_environment_vm.test_vm: Destroying... [id=200]
proxmox_virtual_environment_vm.test_vm: Destruction complete after 9s
proxmox_virtual_environment_vm.ubuntu_template: Destruction complete
proxmox_download_file.ubuntu_image: Destruction complete after 6s

Destroy complete! Resources: 3 destroyed.
```

One expectation to set before the first destroy: the next apply starts with the image download again, because destroying the download resource deletes the qcow2 from the storage, so budget the same five minutes for the rebuild. When the goal is instead to keep the template but forget a single broken resource, terraform state rm followed by the resource address removes it from state without touching the cluster, and the next plan offers to create it anew. The destroy transcript above is expected output rather than a field transcript; the lab dismantled its first stack by hand during the Phase 2 rebuild, and honest labeling of what was measured versus what is predicted is part of the contract this series keeps.

8. Step 6: Verification

Trust nothing until three independent layers agree: the hypervisor, the guest and the network. The verification matrix below takes about five minutes and catches the failure modes that otherwise surface days later, such as a cloud-init drive that exists but was never read by the guest.

8.1 Hypervisor side

RUN ON ANY PVE NODE

```
qm list # vm-test-01 (VMID 200) running
qm config 200 | grep -E "scsi0|net0|agent|serial" # expected hardware
qm cloudinit pending 200 # settings not yet applied
qm guest cmd 200 ping # works after Step 7 installs agent
```

HYPERVISOR CHECKS - FIELD OUTPUT

```
qm list
VMID NAME STATUS MEM(MB) BOOTDISK(GB) PID
9000 ubuntu-2604-tpl stopped 2048 10.00 0
200 vm-test-01 running 2048 10.00 318234

qm config 200 | grep -E "scsi0|net0|agent|serial"
agent: enabled=1
net0: virtio=BC:24:11:2E:AA:F7,bridge=vmbr0
scsi0: nfs-shared:vm-200-disk-0,size=10G
serial0: socket
vga: serial0

qm guest cmd 200 ping
(no output; exit code 0 - silence is the success signal)
```

Three habits are worth forming from this output. The template shows in `qm list` with status stopped and PID 0, exactly as a mould should; anything else means something started the template, which should never happen. The `grep` on `qm config` verifies the four hardware decisions from Chapter 6 at a glance: agent enabled, virtio networking on `vmbr0`, the root disk on `nfs-shared`, and the serial console pair that keeps noVNC usable. And `qm guest cmd ping` prints nothing at all when it succeeds; the command talks to the guest agent over the virtio serial channel, and an absent agent answers with a loud error instead, so silence plus a zero exit code is the pass condition.

The cloudinit pending view deserves a habit-forming note: it shows what cloud-init would change on next boot. Right after provisioning it should mirror the initialization block from `vm.tf`. If it is empty while the cloud-init drive exists, the drive was created but the datasource inside the guest never matched it, and the troubleshooting chapter has the corresponding entry.

8.2 Guest and network side

RUN FROM CTRL-01

```
ping -c2 192.168.122.50 # network layer agrees
ssh ubuntu@192.168.122.50 # key-based login

# inside the VM:
cloud-init status --wait # expect: status: done
sudo tail -n 20 /var/log/cloud-init-output.log
```

FIRST LOGIN AND THE CLOUD-INIT RECEIPT

```
ssh ubuntu@192.168.122.50
The authenticity of host '192.168.122.50' can't be established.
ED25519 key fingerprint is SHA256:dG9w...DzKw.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.122.50' (ED25519) to the list of
known hosts.
Welcome to Ubuntu 26.04 LTS (Resolute) (GNU/Linux 6.17.0-25-generic...)

ubuntu@vm-test-01:~$ cloud-init status --wait
status: done

ubuntu@vm-test-01:~$ grep Datasource /var/log/cloud-init-output.log
Cloud-init v. 25.1 finished at ... Datasource DataSourceNoCloud
seeddev=/dev/sr0. Up 14.32 seconds
```

The transcript holds three separate confirmations. The host key prompt appears exactly once, on the first connection, and answering yes records the VM key in `known_hosts`; a rebuilt VM presents a new key, so reconnecting after a destroy cycle needs `ssh-keygen -R 192.168.122.50` first, a lesson the lab hit during the Phase 2 rebuild. The prompt inside the guest reads `vm-test-01`, which proves the cloud-init hostname took hold even without the removed `initialization.hostname` attribute. And the two closing lines are the datasource receipt: `status: done` plus `DataSourceNoCloud seeddev=/dev/sr0` means the seed drive was found and fully consumed on first boot.

Check	Command	Pass condition
VM running	<code>qm list</code>	status running, correct VMID
Hardware matches	<code>qm config</code>	<code>virtio-scsi-pci</code> , <code>serial0</code> , <code>agent=1</code>
IP applied	<code>ip a</code> (in guest)	static address, no DHCP lease
Cloud-init done	<code>cloud-init status --wait</code>	status: done
User and key	<code>ssh</code> from <code>ctrl-01</code>	login without password
Agent (after Step 7)	<code>qm guest cmd ping</code>	returns without error

Table 6. The six-point verification matrix for a provisioned VM.

One silent failure deserves special attention because it looks like success in the Proxmox interface. The VM can boot, appear as running, and still have received no cloud-init payload at all if the NoCloud datasource found no seed. The symptom is a guest with the image default DHCP identity instead of the configured static address. The fastest check is `cloud-init status` in the guest: it reports the datasource it used, and a healthy provision prints `DataSourceNoCloud seeddev=/dev/sr0`.

9. Step 7: Ansible First Contact

Terraform stops at a booted, networked VM. Everything that happens inside the operating system from here on belongs to Ansible, and Phase 3 will grow this first playbook into the full agent stack: Zabbix Agent 2, Trend Micro, ManageEngine and the rsyslog forwarder to QRadar. The Phase 1 playbook is intentionally small; its job is to prove the management channel and to install the one package the hypervisor itself cares about, the QEMU guest agent.

9.1 Inventory and connectivity

ANSIBLE/INVENTORY/HOSTS.YML

```
all:
  children:
    provisioned:
      hosts:
        vm-test-01:
          ansible_host: 192.168.122.50
          ansible_user: ubuntu
```

RUN INSIDE CTRL-01, FROM THE ANSIBLE/ DIRECTORY

```
ansible provisioned -m ping

vm-test-01 | SUCCESS => {
  "ansible_facts": { "discovered_interpreter_python": "/usr/bin/python3" },
  "changed": false,
  "ping": "pong"
}
```

The ping module does more than its name suggests: it authenticates over SSH with the cloud-injected key, selects a Python interpreter and runs a trivial module on the remote side. A green SUCCESS here means every layer from Chapter 8 is aligned, and it is the moment the pipeline can honestly be called zero-touch: from this point on, nobody logs into a console again.

9.2 The first playbook

ANSIBLE/PLAYBOOKS/01_FIRST_CONTACT.YML

```
---
- name: First contact - verify a freshly provisioned VM
  hosts: provisioned
  become: true
  gather_facts: true

  tasks:
    - name: Show system identity
      ansible.builtin.debug:
        msg: "{{ inventory_hostname }}" is up,
        {{ ansible_facts['distribution'] }}
        {{ ansible_facts['distribution_version'] }}"

    - name: Install QEMU guest agent
      ansible.builtin.apt:
        name: qemu-guest-agent
        state: present
        update_cache: true

    - name: Enable and start the guest agent
      ansible.builtin.service:
        name: qemu-guest-agent
        state: started
        enabled: true

    - name: Mark the VM as managed by this pipeline
      ansible.builtin.copy:
        content: "provisioned-by: terraform + bpg/proxmox\nphase: 1\n"
        dest: /etc/provisioned-by
```

RUN INSIDE CTRL-01

```
ansible-playbook playbooks/01_first_contact.yml
```

THE FIRST RUN, EXACTLY AS THE FIELD PRINTED IT

```
PLAY [First contact - verify a freshly provisioned VM] *****

TASK [Gathering Facts] *****
ok: [vm-test-01]

TASK [Show system identity] *****
ok: [vm-test-01] => {
  "msg": "vm-test-01 is up, Ubuntu 26.04"
}

TASK [Install QEMU guest agent] *****
changed: [vm-test-01]

TASK [Enable and start the guest agent] *****
ok: [vm-test-01]

TASK [Mark the VM as managed by this pipeline] *****
changed: [vm-test-01]

PLAY RECAP *****
vm-test-01 : ok=5 changed=2 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

The recap line deserves arithmetic, because `ok=5 changed=2` looks wrong until the convention is known: the `ok` counter includes every successful task, `changed` ones among them. Five task results ran in total, and three of them would normally report `changed`, but the service task reports `ok` on a fresh VM because the Ubuntu package `postinst` already enabled and started `qemu-guest-agent` a second earlier, during the `apt` task. The two `changed` results are therefore the package install and the marker file, and the field run printed exactly this line. When the counts drift from this shape on a fresh VM, something upstream already touched the guest; rerun with `-v` to see which task behaved differently.

THE SECOND RUN, ONE MINUTE LATER - THE IDEMPOTENCE RECEIPT

```
PLAY RECAP *****
vm-test-01 : ok=5 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

Running the playbook a second time is not busywork; it is the idempotence receipt. Every task now reports `ok`, because the package is present, the service is running and enabled, and the marker file already carries the exact content, so `changed` falls to zero. That property is what turns the playbook from a one-shot installer into a safe recurring operation: Phase 3 schedules this same pattern against every provisioned VM, where a `changed` count above zero becomes a drift signal worth investigating instead of a routine side effect.

The fact references deliberately use the `ansible_facts` bracket syntax rather than the bare `ansible_distribution` style most older playbooks use. `ansible-core 2.20` deprecates automatic fact injection with a warning on every play (it disappears in 2.24), so writing `ansible_facts` now simply moves the fix before the breakage. The first run on a freshly cloned VM reports two `changed` tasks - the agent package install and the marker file - and zero failures, which is the signal that the management channel from

ctrl-01 all the way into the guest works.

The guest agent is the piece that closes the loop with the hypervisor: once it runs, the Proxmox interface shows the VM IP address, qm guest commands work for clean shutdowns and filesystem freezes, and monitoring gets accurate guest information. Installing it through Ansible rather than cloud-init keeps the user-data minimal and puts the package under proper configuration management, where a weekly drift run can detect if someone removed it by hand. The final task writes a small marker file; it costs nothing and answers the question every operator eventually asks about an unfamiliar machine: where did you come from.

10. Troubleshooting Matrix

Every failure below was either observed in the lab or documented in the upstream issue trackers of the tools involved. The rightmost column is the fix, not a workaround; the goal is that each row ends with a re-verification step from Chapter 8. Proxmox task logs (node view, Task History) and the guest file `/var/log/cloud-init-output.log` resolve the remaining cases that no table can predict.

Symptom	Cause	Fix
Guest crash-loops early at boot; console shows "kernel requires x86-64-v3" or a glibc ISA error	amd64v3 image booted under a v2 virtual CPU (PVE default x86-64-v2-Auto)	Set <code>cpu_type = "x86-64-v3"</code> (or "host" on the nested lab) and re-apply; or switch <code>cloud_image_url</code> to the baseline amd64 image
Download task runs 10 minutes then fails: "timeout while waiting for task ... and the content type 'import' is not supported by the Proxmox VE version"	PVE below 8.4 has no import content type (bpg provider gates it at 8.4.0); the 600 s default <code>upload_timeout</code> is also tight for slow links	Upgrade the node to PVE 8.4+ or 9.2 (Chapter 7.2); the repo sets <code>upload_timeout = 3600</code> ; re-apply
Download fails with "wrong file extension" on a .qcow2 or .raw image name	PVE below 8.4 rejects those extensions in the download-url API	Upgrade to 8.4+, or fall back to <code>content_type = "iso"</code> with an .img file name (that path then needs the provider ssh block, see next row)
Template creation fails with "creating custom disk: ... ssh" / SSH connection error while using disk <code>file_id</code>	The <code>file_id</code> import path SSHes into the PVE node and runs qm disk import; API tokens cannot SSH	Use disk <code>import_from</code> (PVE 8.4+, as shipped in this repo), or add an ssh {} block with node credentials for the legacy <code>file_id</code> path
terraform apply fails on <code>proxmox_download_file</code> with HTTP 500	Import content type not enabled on the NFS storage	<code>pvesm set nfs-shared -content images,rootdir,vztmpl,snippets,import</code> , then re-apply

Symptom	Cause	Fix
terraform plan: "Unsupported argument ... hostname is not expected here" (initialization block)	Provider v0.111 removed initialization.hostname; the cloud-init hostname now comes from the VM name	Delete the hostname line; keep the VM name a valid DNS name
terraform plan: "invalid value for type" listing CPU models, got x86-64-v3-Auto	The provider validator accepts x86-64-v3 but not the PVE -Auto variants	Set cpu_type = "x86-64-v3" (migration-safe) or "host" on nested-virt labs
terraform plan: "Unsupported argument ... vlan_tag is not expected here"	Provider v0.111 renamed the network_device attribute; older tutorials use vlan_tag	Use vlan_id = N inside network_device (0 or omitted = untagged); trunks = "10;20;30" is the separate VLAN-aware bridge feature
ansible --version fails with "could not initialize the preferred locale"	Template sets LANG=en_US.UTF-8 but never generates the locale	apt-get install locales, locale-gen en_US.UTF-8, update-locale LANG=en_US.UTF-8 (Chapter 5.2)
apt-get update rejects the HashiCorp repository as unsigned (NO_PUBKEY)	Minimal container lacks gpg, so the dearmor pipe never created the keyring	Store the armored key as hashicorp.asc (Chapter 5.2) or install gnupg first, then apt-get update again
Provider error 401/403 on every operation	Token value malformed (whitespace) or privileges missing	Re-copy the token exactly; verify with pveum acl list; the token needs the user privileges from Step 2
Error "x509: certificate signed by unknown authority"	Self-signed PVE certificate and insecure not set	Set pve_tls_insecure = true for the lab, or install the proper CA chain for production
VM boots with DHCP address instead of the configured static IP	NoCloud datasource never found the seed drive	Verify qm cloudinit pending shows the settings; ensure the cloud-init drive exists (ide2) and serial console is attached
ansible ping fails with "Permission denied (publickey)"	SSH key not injected, or wrong username	Check user_account.keys in vm.tf and the vm_username value; re-run apply so cloud-init drive regenerates
Console in noVNC shows a blank or frozen screen	Standard VGA attached while the image writes to the serial port	Keep serial_device plus vga serial0 as in the repo template
Clone apply is slow or times out	NFS latency while copying the template disk	Keep clone retries = 3; for faster labs consider linked clones (Phase 5 topic)
Apply or plan stalls about 15 minutes on a running VM, then: Warning: error waiting for network interfaces from QEMU agent	agent enabled in the config makes the provider poll the guest agent after start and on every refresh; Ubuntu cloud images ship without the agent (default agent.timeout = 15m)	Not an error - state is clean, VM runs. The repo ships agent wait_for_ip { disabled = true } (Section 7.3); after Chapter 9 installs the agent, remove the block to populate ipv4_addresses

Symptom	Cause	Fix
Mid-session SSH to the node dies with Connection reset by peer; afterwards the control container briefly shows not running and the workload VM is stopped	The whole lab froze with its host: in a nested-virt setup the PVE nodes are themselves guests of a laptop, and host sleep suspends every guest at once (field incident: laptop lid, lunch break)	Nothing crashed. Wake the host and guests resume; a VM that stayed stopped restarts on the next terraform apply (plan shows started false to true). Disable host auto-suspend during lab hours; confirm with uptime and journalctl --list-boots

Table 7. Field failures, their causes, and the fixes that hold.

A general debugging rule for this pipeline: change one variable at a time and re-run terraform plan between changes. The provider diff is accurate about what will change, and the plan output frequently reveals that a fix touched more resources than intended, for example a CPU type change that forces a template rebuild and therefore re-clones every workload that references it.

11. Production Notes and Phase 2 Preview

11.1 From homelab to production

The pipeline built in this guide is production-shaped but not yet production-hardened. The deltas below are the honest list of what changes when the same code moves to a customer environment, and each one is a deliberate Phase 5 work item rather than an afterthought. None of them invalidates the Phase 1 design; they extend it.

Homelab shortcut	Production reality
Administrator role on / for the token	Custom role with the minimum privilege set from Chapter 4
Self-signed certificate with insecure = true	Internal CA chain, insecure = false, certificate rotation
Local terraform.tfvars and local state	CI variables plus a remote state backend with locking
NAT bridge, no VLANs	802.1Q trunks; set vlan_tag per environment segment
Manually chosen static IPs	IPAM allocation (NetBox) feeding the pipeline variables
One VM per apply	Controlled parallelism; batch templates and naming schemes

Table 8. The honest homelab-to-production delta list.

11.2 What Phase 2 adds

Phase 1 provisions a single-disk VM, which covers web servers and jump hosts but not database servers. The next module in the series addresses the multi-disk pattern: an OS disk plus separate volumes for /var, /log, /data and /backup, each living on its own SCSI device. The foundations laid here are exactly the ones Phase 2 builds on. The virtio-scsi-pci controller chosen in Chapter 6 is the controller that supports online disk resize; the serial attribute of the provider disk block (a string of up to 20 bytes) will give every data disk a stable identity that survives device letter reshuffling inside the guest through the /dev/disk/by-id path.

The Phase 2 playbook chain will then be mechanical: growpart on the partition, pvresize on the physical volume, lvextend against the volume group, and xfs_growfs on the filesystem, all idempotent and all triggered by nothing more than a size change in the Terraform configuration. When a customer asks for a terabyte of additional data space, the answer becomes a one-line variable edit and a re-apply, which is the standard this series is working toward: infrastructure that answers requests in minutes, with an audit trail, and without a single manual console session.